AXI4-Lite Cache

Microarchitectural Specification

Doc Identifier:	AXI4LITE-CACHE	**Release Date:**	October 09, 2026
Version:	Rev 1.0	**Status:**	Review
Author:	Danil Zavarnitsyn	**Reviewer:**	Maria Stepanova

Contents

1 Introduction **2**
1.1 Characteristics . . . 2

2 Functional Description **3**
2.1 Request Fetching . . . 3
2.2 Read Operation . . . 4
2.2.1 Read Hit . . . 4
2.2.2 Read Miss . . . 4
2.3 Write Operation . . . 4
2.3.1 Write Hit . . . 4
2.3.2 Write Miss . . . 5
2.4 Hits Counting . . . 5

3 Clock and Reset **5**

4 Parameters **5**

5 Ports Description **6**

6 Submodules **7**
6.1 Comparator . . . 7
6.2 Evicting Block . . . 8
6.3 Pseudo-Random Generator . . . 9
6.4 Data Memory . . . 10
6.5 Tag Memory . . . 11
6.6 Arbiter . . . 12
6.7 Buffer . . . 13
6.8 Statistics Block . . . 13

1 Introduction

The AXI4-Lite Cache is a parameterizable set-associative cache designed to reduce the average memory access latency between a processor and an external memory system.

1.1 Characteristics

This subsection describes the main features and capabilities supported by the AXI4-Lite Cache.

Characteristic	Description
Configurable Associativity	Configurable number of cache ways.
Configurable Address and Data Widths	Configurable address and data widths.
Configurable Number of Sets	Configurable number of cache sets.
Write-Through Policy with no-allocation	Writes are immediately propagated to external memory.
NMRU Replacement Policy	Approximation of LRU based on the Not Most Recently Used (NMRU) policy.
Data-Width Cache Line	Cache line size is equal to the data width.
AXI4-Lite Interface	AXI4-Lite interfaces on the CPU and external memory sides.
No Outstanding Transactions	Only one cache transaction can be processed at a time..
Hit/Miss Statistics	Hit and miss counters are available during simulation only.

Table 1: Cache characteristics

2 Functional Description

This section describes the cache operation for read and write requests, including cache hits, cache misses, and interactions with external memory.

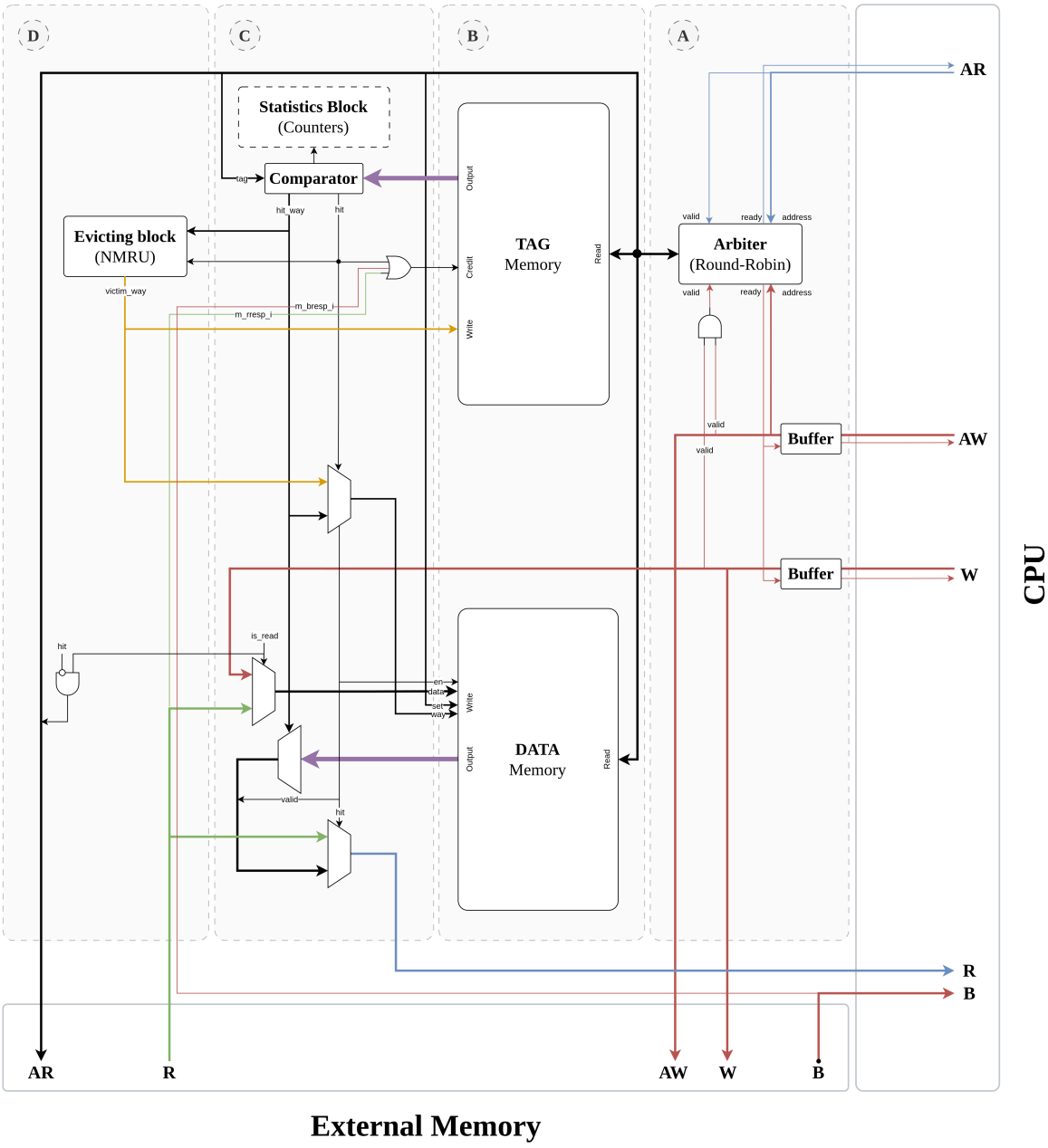


Figure 1: Cache scheme

2.1 Request Fetching

All incoming CPU transactions are initially received by Buffers located before the Arbiter. Separate Buffers are provided for the `AR`, `AW`, and `W` channels (see Ⓐ).

The Buffers serve two purposes.

- First, they temporarily hold transactions when read and write requests arrive simultaneously, allowing the Arbiter which operation should be processed first.
- Second, they account for the fact that the AW and W channels of an AXI4-Lite write transaction may arrive independently and at different times.

A write operation is passed to the cache for processing only after both the AW and W transactions have been received from the CPU.

If only one type of transaction is received, either a read request through the AR channel or a complete write request with both AW and W transactions, it passes through the corresponding buffers and Arbiter combinatorially.

2.2 Read Operation

During a read operation, the CPU provides the requested address through the AXI4-Lite AR channel. The address is used simultaneously to initiate a read operation in both the Tag Memory and Data Memory (see Ⓑ).

The set index is extracted from the requested address and is used to access the corresponding set in both memories. On the following clock cycle, the Tag Memory provides the tags and valid bits of all ways belonging to the selected set. In parallel, the Data Memory provides the data stored in all ways of the same set.

The set of ways obtained from the Tag Memory, together with the requested tag extracted from the address, is provided to the Comparator. The Comparator compares the requested tag with the tag stored in each way in parallel and determines whether the requested cache line is present (see Ⓒ).

2.2.1 Read Hit

In case of a cache hit, the Comparator asserts the hit signal and provides the index of the way containing the requested tag through hit_way.

The hit_way value is provided to the Evicting Block to update the most recently used way for the corresponding set. At the same time, hit_way is used to select the required data from the set of ways returned by the Data Memory. The selected data is then provided to the CPU through the AXI4-Lite R channel (see Ⓒ).

Since the Tag Memory and Data Memory are accessed in parallel, the cache can determine the requested way and select the corresponding data within a single clock cycle.

2.2.2 Read Miss

In case of a cache miss, the Evicting Block is activated to select the way that will be replaced in the selected set. The victim way is selected by Pseudo-Random Generator while ensuring that the most recently used way is not selected for replacement (see Ⓓ). The generated victim way is then used to select the cache way that will be overwritten in the Tag Memory.

At the same time, the address of the requested cache line is provided to the external memory through the AXI4-Lite AR channel. The data returned by the external memory on the R channel is written into the selected way of the Data Memory and is simultaneously forwarded to the CPU through the CPU-side AXI4-Lite R channel (see Ⓒ).

2.3 Write Operation

During a write operation, the data is written directly to the external memory through the AXI4-Lite interface and allocates in the cache according to the write-through policy.

When the CPU provides the write address and data through the AXI4-Lite AW and W channels, the write address is simultaneously provided to the Tag Memory (see Ⓑ). The set index extracted from the address is used to access the corresponding set, and the requested tag is compared with the tags of all ways by the Comparator, following the same procedure as for a read operation (see Ⓒ).

2.3.1 Write Hit

If a matching tag is found, the corresponding way is identified by hit_way. The new data is written both to the selected way of the Data Memory (see Ⓒ) and to the external memory. The write address and data received from the CPU through the AXI4-Lite AW and W channels are forwarded to the corresponding channels of the external memory interface.

2.3.2 Write Miss

If no matching tag is found, the Evicting Block is activated to select the way that will be replaced in the selected set (see Ⓓ). The victim way is selected by Pseudo-Random Generator while ensuring that the most recently used way is not selected for replacement.

The tag of the current write address is written into the selected way of the Tag Memory, and the corresponding data is written into the selected way of the Data Memory (see Ⓒ). At the same time, the write address and data received from the CPU through the AXI4-Lite `AW` and `W` channels are forwarded to the corresponding channels of the external memory interface.

2.4 Hits Counting

The cache provides hit and miss counting functionality for simulation and performance analysis. This functionality is implemented in the Statistics Block.

The Statistics Block receives hit and miss information from the Comparator, which determines whether the requested tag is present in the selected cache set.

Simulation Only

The Statistics Block is intended exclusively for simulation purposes and is not included in the synthesized cache design. Therefore, it does not introduce any hardware overhead in the final implementation.

3 Clock and Reset

The AXI4-Lite Cache operates within a single clock domain and a single reset domain. No clock domain crossings are present. The cache uses the `clk_i` signal as its clock and the `rstn_i` signal as an active-low synchronous reset.

During reset, the entire Tag Memory is cleared, including all stored tags and valid bits. The contents of the Data Memory do not need to be reset, as they are considered invalid until a corresponding valid entry is created in the Tag Memory.

4 Parameters

The following table lists the parameters used to configure the AXI4-Lite Cache implementation.

Parameter	Valid Range	Default	Description
`ADDR_WIDTH`	Positive integer	`32`	Width of the memory address.
`DATA_WIDTH`	32, 64	`32`	Width of the data bus.
`SET_NUM`	Power of 2	`4`	Number of cache sets.
`WAY_NUM`	Power of 2	`4`	Number of cache ways (associativity).
`DEBUG_ON`	0, 1	`0`	Enables the Statistics Block when set to `1`.

Table 2: RTL configuration parameters

The following auxiliary parameters are introduced for documentation purposes only. They are used to simplify the description of signal widths and cache organization and do not represent parameters of the RTL implementation.

Parameter	Definition	Description
`SET_WIDTH`	$\log_2(\texttt{SET_NUM})$	Width of the cache set index.
`WAY_WIDTH`	$\log_2(\texttt{WAY_NUM})$	Width of the way index.

Table 3: Auxiliary documentation parameters

5 Ports Description

The cache acts as an AXI4-Lite slave on the CPU side and as an AXI4-Lite master on the external memory side. The following table describes all ports of the cache and their respective directions, widths, and functions.

Ports	Direction	Width	Description
clk_i	Input	1	System clock used to synchronize the cache operation.
rstn_i	Input	1	Active-low synchronous reset signal.
CPU-side AXI4-Lite interface			
s_axi_awaddr_i	Input	ADDR_WIDTH	Write address provided by the CPU.
s_axi_awvalid_i	Input	1	Indicates that a valid write address is available.
s_axi_awready_o	Output	1	Indicates that the cache is ready to accept a write address.
s_axi_wdata_i	Input	DATA_WIDTH	Write data provided by the CPU.
s_axi_wstrb_i	Input	DATA_WIDTH / 8	Write byte strobes indicating which bytes of the write data are valid.
s_axi_wvalid_i	Input	1	Indicates that valid write data is available.
s_axi_wready_o	Output	1	Indicates that the cache is ready to accept write data.
s_axi_bresp_o	Output	2	Write response indicating the result of the write transaction.
s_axi_bvalid_o	Output	1	Indicates that a valid write response is available.
s_axi_bready_i	Input	1	Indicates that the CPU is ready to accept the write response.
s_axi_araddr_i	Input	ADDR_WIDTH	Read address provided by the CPU.
s_axi_arvalid_i	Input	1	Indicates that a valid read address is available.
s_axi_arready_o	Output	1	Indicates that the cache is ready to accept a read address.
s_axi_rdata_o	Output	DATA_WIDTH	Read data returned to the CPU.
s_axi_rresp_o	Output	2	Read response indicating the result of the read transaction.
s_axi_rvalid_o	Output	1	Indicates that valid read data is available.
s_axi_rready_i	Input	1	Indicates that the CPU is ready to accept the read data.
External memory-side AXI4-Lite interface			
m_axi_awaddr_o	Output	ADDR_WIDTH	Write address provided to the external memory.
m_axi_awvalid_o	Output	1	Indicates that a valid write address is available.
m_axi_awready_i	Input	1	Indicates that the external memory is ready to accept a write address.
m_axi_wdata_o	Output	DATA_WIDTH	Write data provided to the external memory.
m_axi_wstrb_o	Output	DATA_WIDTH / 8	Write byte strobes indicating which bytes of the write data are valid.
m_axi_wvalid_o	Output	1	Indicates that valid write data is available.
m_axi_wready_i	Input	1	Indicates that the external memory is ready to accept write data.
m_axi_bresp_i	Input	2	Write response returned by the external memory.
m_axi_bvalid_i	Input	1	Indicates that a valid write response is available.
m_axi_bready_o	Output	1	Indicates that the cache is ready to accept the write response.
m_axi_araddr_o	Output	ADDR_WIDTH	Read address provided to the external memory.
m_axi_arvalid_o	Output	1	Indicates that a valid read address is available.
m_axi_arready_i	Input	1	Indicates that the external memory is ready to accept a read address.
m_axi_rdata_i	Input	DATA_WIDTH	Read data returned by the external memory.
m_axi_rresp_i	Input	2	Read response returned by the external memory.
m_axi_rvalid_i	Input	1	Indicates that valid read data is available.
m_axi_rready_o	Output	1	Indicates that the cache is ready to accept the read data.

Table 4: Cache ports

6 Submodules

This section describes the micro-architecture and functionality of the submodules used in the cache implementation. For each module, its purpose, internal organization, inputs and outputs, and interaction with other components are presented.

6.1 Comparator

The Comparator module is responsible for determining whether the requested tag is present in the selected cache set. The module receives the requested tag and the array of tags corresponding to all ways of the selected set from the Tag Memory.

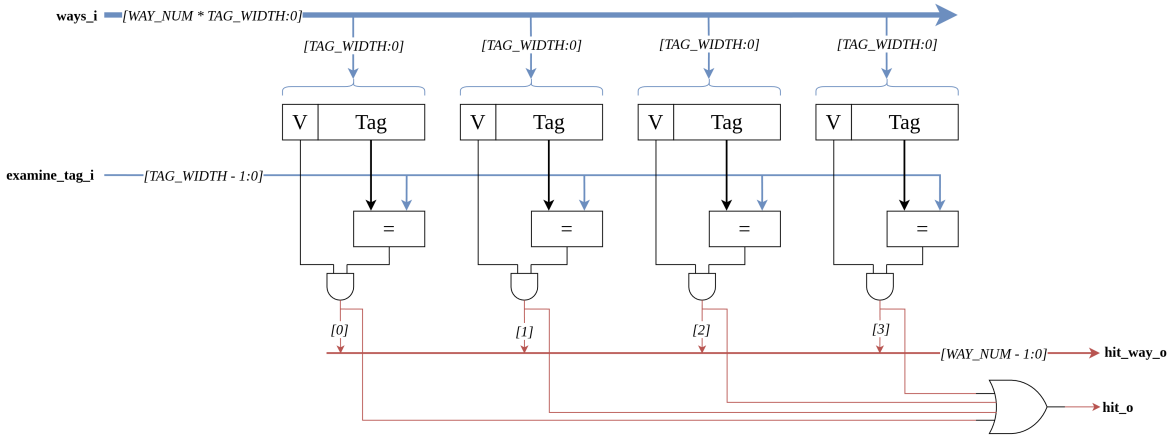


Figure 2: Comparator scheme (4-way cache)

The comparison is performed in parallel. For each way, a dedicated comparator compares the tag stored in the corresponding way with the requested tag. The results of the individual comparisons form a match vector. Each bit of this vector corresponds to one way and is asserted when the tag stored in the corresponding way matches the requested tag. Thus, the output of the module represents the location of the matching tag using a one-hot encoding.

Parameter	Valid Range	Default	Description
`TAG_WIDTH`	Positive integer	—	Width of the cache tag.
`WAY_NUM`	Positive integer	—	Number of cache ways compared in parallel.

Table 5: Comparator parameters

Ports	Direction	Width	Description
`ways_i`	Input	`WAY_NUM * (TAG_WIDTH + 1)`	Tags and valid bits of all ways belonging to the selected set.
`examine_tag_i`	Input	`TAG_WIDTH`	Tag of the requested cache line to be compared against all ways.
`hit_way_o`	Output	`WAY_NUM`	One-hot encoded indication of the way containing the requested tag..
`hit_o`	Output	`1`	Indicates whether the requested tag was found in any way of the selected set.

Table 6: Comparator ports

6.2 Evicting Block

The Evicting Block is responsible for selecting a victim way when a cache miss occurs and a new cache line has to be allocated. The module implements the *Not Most Recently Used* (NMRU) replacement policy.

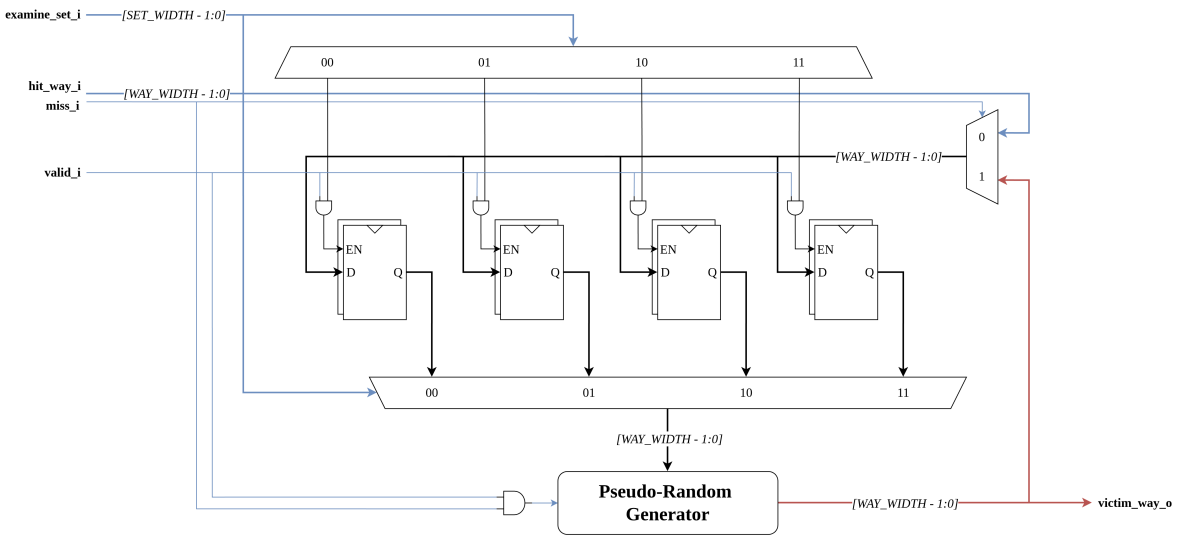


Figure 3: Evicting Block scheme (4-set cache)

Parameter	Valid Range	Default	Description
`SET_NUM`	Positive integer	—	Number of cache sets for which the replacement state is maintained.
`WAY_NUM`	Positive integer	—	Number of cache ways used for victim selection.

Table 7: Evicting Block parameters

Ports	Direction	Width	Description
`examine_set_i`	Input	`SET_WIDTH`	Index of the current cache set.
`hit_way_i`	Input	`WAY_WIDTH`	Index of the way containing the requested tag on a cache hit.
`miss_i`	Input	`1`	Indicates that the current cache access resulted in a cache miss.
`valid_i`	Input	`1`	Indicates whether the selected cache line contains valid data.
`victim_way_o`	Output	`WAY_WIDTH`	Index of the way selected for replacement on a cache miss.

Table 8: Evicting Block ports

The module maintains a separate register for each cache set. Each register stores the index of the way that was most recently used within the corresponding set.

When a cache access results in a hit, the way containing the requested tag becomes the most recently used way.

When a cache miss occurs, the module receives the index of the selected set and reads the corresponding MRU (Most Recently Used) register. The stored way index is then used as a constraint for victim selection: the victim way must not be equal to the most recently used way.

To select the victim, the module contains a Pseudo-Random Generator. The generator is activated on a cache miss and produces a candidate way index.

Once a valid victim way has been generated, its index is driven to the module output and is used by the cache control logic to select the way that will be replaced. The selected victim way also becomes the new most recently used way. Therefore, the current MRU register is updated with the generated victim index. This ensures that the most recently accessed or allocated way is excluded from being selected as the victim during the next replacement operation for the same set.

6.3 Pseudo-Random Generator

The Pseudo-Random Generator is responsible for generating a candidate (victim way) used by the Evicting Block during cache line replacement. The generator is based on a *Linear Feedback Shift Register* (LFSR), which provides a compact hardware implementation for generating pseudo-random sequences.

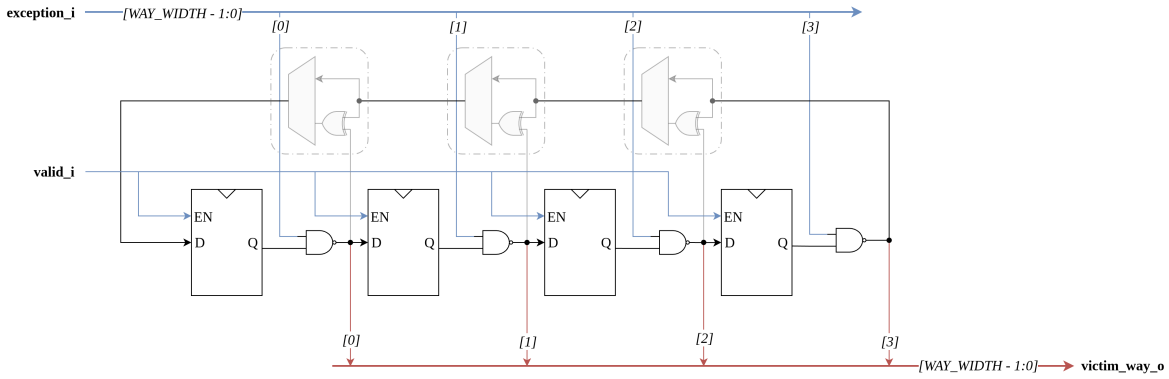


Figure 4: Pseudo-Random Generator scheme

Parameter	Valid Range	Default	Description
LFSR_WIDTH	Positive integer	WAY_WIDTH	Number of flip-flops in the LFSR.
LFSR_POLY	Configurable	—	Selects the feedback polynomial used by the LFSR.

Table 9: LFSR configuration parameters

Ports	Direction	Width	Description
exception_i	Input	WAY_WIDTH	Way index that must be excluded from the generated victim way.
valid_i	Input	1	Indicates whether the selected exception way is valid.
victim_way_o	Output	WAY_WIDTH	Pseudo-randomly generated way index selected as the victim way.

Table 10: Pseudo-Random Generator ports

For a cache with WAY_NUM ways, the minimum LFSR width is

$$N = \log_2(\ \texttt{WAY_NUM}\ ) \tag{1}$$

flip-flops. The feedback function of the LFSR is configurable, allowing different feedback polynomials to be selected. The polynomial configuration is implemented using multiplexers placed between adjacent stages of the shift register. For the i-th polynomial coefficient, the corresponding multiplexer determines whether the feedback path contains an XOR operation or a direct connection.

The NAND-based transformation provides a simple mechanism for modifying the generated value when it coincides with the prohibited way. If a generated bit matches the corresponding bit of the prohibited way, the NAND logic inverts the affected bit. Therefore, if the complete generated victim way coincides with the prohibited way, the corresponding matching bits are inverted, resulting in a different way index.

The NAND-based feedback also prevents the LFSR from becoming permanently locked in the all-zero state. In a conventional LFSR, the state consisting entirely of zeros is a lock-up state: since all feedback values remain zero, the register cannot transition to another state. In the proposed design, the feedback logic additionally depends on the prohibited way index. Even when the generated way and the prohibited way do not completely match, any matching individual bits can still be inverted by the NAND logic. Consequently, when the LFSR reaches the all-zero state, the interaction with the prohibited way can introduce non-zero bits into the next LFSR state. Therefore, the generator can leave the conventional all-zero lock-up state without requiring separate zero-state recovery logic.

Victim Way Distribution

The NMRU replacement policy relies on the pseudo-random generator providing a sufficiently uniform distribution of victim ways. Ideally, each possible way index should have approximately the same probability of being generated.
This property is not guaranteed by the proposed generator architecture. In the proposed implementation, additional NAND logic is introduced between the LFSR stages to account for the prohibited way. This modification changes the state transition function of the original LFSR and can therefore introduce a bias into the distribution of generated victim ways.
Therefore, before configuring the cache parameters (`SET_NUM`, `WAY_NUM`), the distribution of the generated victim way values must be verified for the selected LFSR configuration. The verification should include a sufficiently long generated sequence and should confirm that all valid way indices occur with approximately equal frequency.
If a significant bias is observed, the distribution can be adjusted by selecting a different feedback polynomial. Another possible solution is to increase the length of the LFSR.

6.4 Data Memory

The Data Memory block encapsulates the multi-bank memory used to store cache data. Each bank stores the data corresponding to a specific way across all cache sets.

Each memory bank is implemented as a pseudo-dual-port memory with a single memory access port that can be used for either a read or a write operation. Read requests are issued by the CPU-side cache logic, while write requests are generated from the R-channel of the external memory-side AXI4-Lite interface. The access type is selected according to the current cache operation.

When a read request is received, the corresponding set index is used to address each memory bank. On the following clock cycle, the Data Memory block provides the data stored in all ways of the selected set.

Both read and write operations have a latency of one clock cycle.

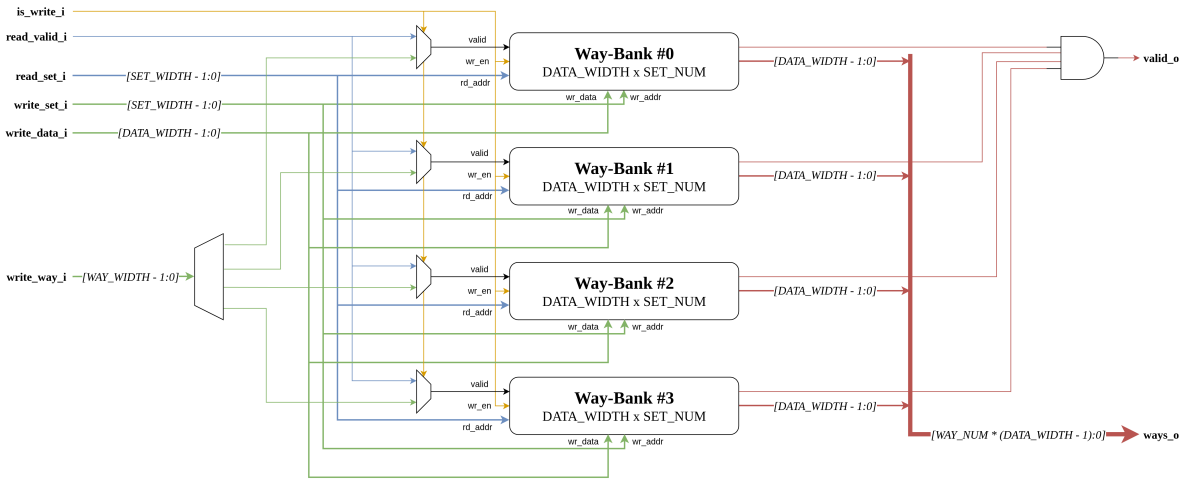


Figure 5: Data Memory scheme (4-way cache)

Parameter	Valid Range	Default	Description
`DATA_WIDTH`	Positive integer	32	Width of the data stored in each cache line.
`WAY_NUM`	Power of 2	—	Number of cache ways.
`SET_NUM`	Power of 2	—	Number of cache sets.

Table 11: Data Memory parameters

Ports	Direction	Width	Description
`read_valid_i`	Input	`1`	Indicates a valid read request.
`read_set_i`	Input	`SET_WIDTH`	Index of the cache set to be read.
`write_set_i`	Input	`SET_WIDTH`	Index of the cache set to which the data is written.
`write_data_i`	Input	`DATA_WIDTH`	Data to be written to the selected cache line.
`write_way_i`	Input	`WAY_WIDTH`	Index of the way in which the input data is stored.
`valid_o`	Output	`1`	Indicates that valid output data.
`ways_o`	Output	`DATA_WIDTH * WAY_NUM`	Data stored in all ways of the requested cache set.

Table 12: Data Memory ports

6.5 Tag Memory

The Tag Memory block encapsulates the multi-bank memory used to store cache tags and valid bits. Each bank stores the tag and valid bit corresponding to a specific way across all cache sets.

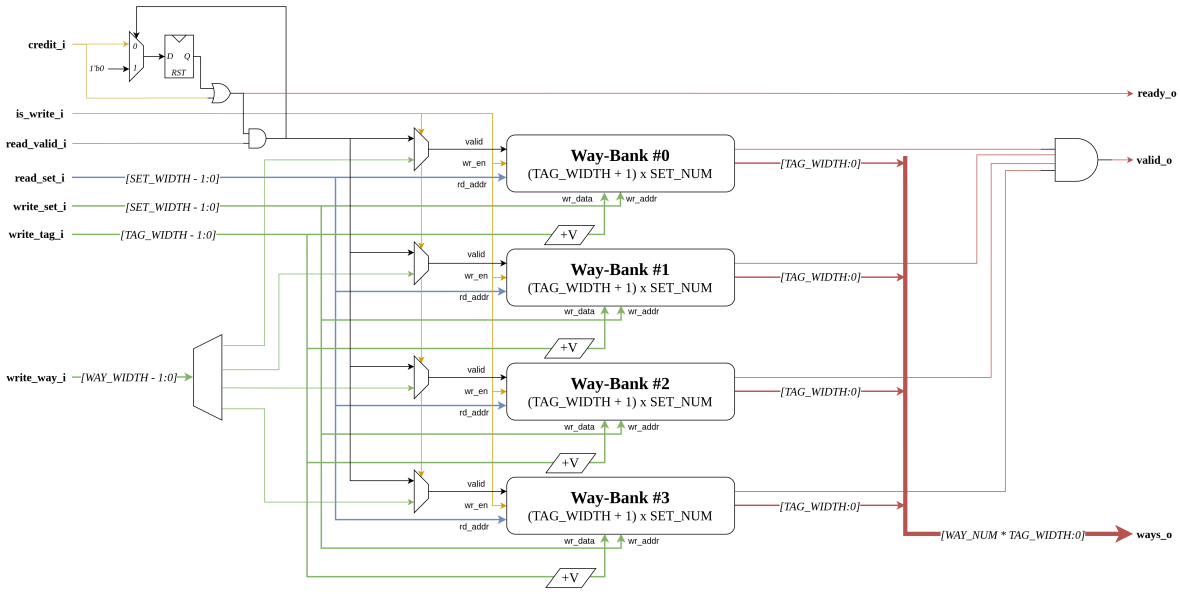


Figure 6: Tag Memory scheme (4-way cache)

Each memory bank is implemented as a pseudo-dual-port memory with a single memory access port that can be used for either a read or a write operation. Read requests are issued by the CPU-side cache logic, while write requests are generated by the cache control logic during cache operations. The access type is selected according to the current cache operation.

The Tag Memory also contains a credit counter that stores a single credit. A read request consumes the available credit, preventing a new read operation from being started until the current cache access has been completed. The credit is restored when the requested data is found in the cache or when a valid response is received from the external memory through the AXI4-Lite `R` or `B` channel. A restored credit can be used in the same clock cycle.

Write operations do not consume the credit and can therefore be performed independently of the credit counter.

Both read and write operations have a latency of one clock cycle.

Parameter	Valid Range	Default	Description
TAG_WIDTH	Positive integer	—	Width of the tag stored in each cache line.
WAY_NUM	Power of 2	—	Number of cache ways.
SET_NUM	Power of 2	—	Number of cache sets.

Table 13: Tag Memory parameters

Ports	Direction	Width	Description
read_valid_i	Input	1	Indicates a valid read request.
read_set_i	Input	SET_WIDTH	Index of the cache set to be read.
write_set_i	Input	SET_WIDTH	Index of the cache set to which the tag is written.
write_tag_i	Input	TAG_WIDTH	Tag to be written to the selected cache line.
write_way_i	Input	WAY_WIDTH	Index of the way in which the tag is stored.
credit_i	Input	1	Indicates that a credit is available for a read request.
ready_o	Output	1	Indicates readiness for a read request accept.
valid_o	Output	1	Indicates that valid output data is available.
ways_o	Output	(TAG_WIDTH + 1) * WAY_NUM	Tags and valid bits stored in requested cache set.

Table 14: Tag Memory ports

6.6 Arbiter

The Arbiter controls access to the memory interface shared by read and write requests. It implements a round-robin arbitration policy.

The arbiter maintains a priority state indicating which request type has priority when read (channel a) and write (channel b) requests are asserted simultaneously. When only one request is active, that request is granted regardless of the current priority. When both requests are active, the request with the current priority is granted. After a request is granted, the priority is updated to favor the other request type.

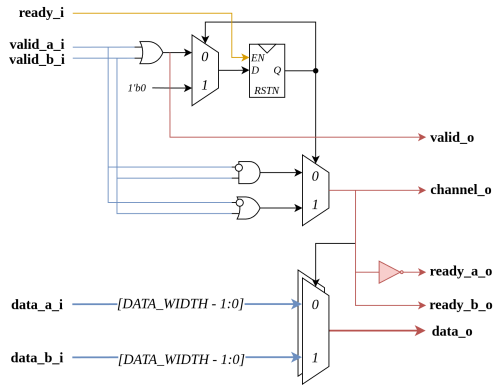


Figure 7: Round-Robin arbiter scheme

Parameter	Valid Range	Default	Description
DATA_WIDTH	Positive integer	32	Width of the transaction data path.

Table 15: Round-Robin Arbiter parameters

Ports	Direction	Width	Description
ready_i	Input	1	Indicates that the downstream logic is ready to accept the selected transaction.
valid_a_i	Input	1	Indicates that a valid transaction is available on channel A.
data_a_i	Input	DATA_WIDTH	Transaction data provided on channel A.
valid_b_i	Input	1	Indicates that a valid transaction is available on channel B.
data_b_i	Input	DATA_WIDTH	Transaction data provided on channel B.
ready_o	Output	1	Indicates that arbiter is ready.
valid_o	Output	1	Indicates that a valid transaction is available at the output.
data_o	Output	DATA_WIDTH	Transaction data selected by the arbiter.

Table 16: Round-Robin Arbiter ports

6.7 Buffer

Buffer is a FIFO with a bypass path used to temporarily store incoming transactions. When the buffer is empty, a valid transaction can bypass the storage element and be passed directly to the output without additional latency. If the output is not ready to accept the transaction, it is stored in the FIFO and remains there until it can be forwarded.

Parameter	Valid Range	Default	Description
DATA_WIDTH	Positive integer	32	Width of the transaction data stored in the buffer.
FIFO_DEPTH	Positive integer	1	Maximum number of transactions that can be stored in the FIFO.

Table 17: Buffer parameters

Ports	Direction	Width	Description
valid_i	Input	1	Indicates that a valid transaction is available at the input.
ready_o	Output	1	Indicates that the buffer can accept a new transaction.
data_i	Input	DATA_WIDTH	Transaction data to be stored in the buffer.
valid_o	Output	1	Indicates that a valid transaction is available at the output.
ready_i	Input	1	Indicates that the downstream logic is ready to accept a transaction.
data_o	Output	DATA_WIDTH	Transaction data provided at the output.

Table 18: Buffer ports

6.8 Statistics Block

The Statistics Block provides cache hit and miss statistics for simulation and performance analysis. It receives hit and miss information from the Comparator and maintains separate counters for cache hits and misses.

The block is enabled by the DEBUG_ON parameter. When DEBUG_ON is set to 0, the block is disabled and does not affect the cache implementation.

Parameter	Valid Range	Default	Description
WIDTH	Positive integer	32	Width of the registers used to store the number of cache hits and misses.

Table 19: Statistics Block parameters

Ports	Direction	Width	Description
valid_i	Input	1	Indicates that the current cache access is valid.
hit_i	Input	1	Indicates whether the current cache access resulted in a hit.
hit_count_o	Output	WIDTH	Number of recorded cache hits.
miss_count_o	Output	WIDTH	Number of recorded cache misses.

Table 20: Statistics Block ports